Token-Saving Guide: Claude Code, Codex, Antigravity, Cursor

Oct 20, 2026 · @Aalok Shah

Your usage lasts longest when each turn carries little context, the cheapest adequate model does the work, and you add only the few tools that independent tests show actually save money. This guide sets that up for all four tools, from day-0 install to the commands and habits for working and taking breaks.

What decides how long your usage lasts

Three things decide it: how much context each turn re-sends, which model does the work, and how many turns and agents you run. Add-on tools that compress output move the bill far less than their claims, and several popular ones make it worse.

Fact	Evidence	What it means for you
Every turn re-sends the whole conversation	Calls with over 150K tokens of context were 74.7% of one measured week that hit a Max 20x limit (dotclaude)	Short sessions, `/clear` between tasks, auto-compact well before the window is full
Model choice is the biggest lever	Switching to a cheaper model was the only repeated cost drop across eight benchmarks, about −49% (catalogue); Claude Code starts on Opus by default since 22 Sep 2026	Set a cheap default; escalate only when a task needs it
Thinking is billed as output	Anthropic's cost guide; one creator measured max effort at 3.5–4x default tokens	Medium effort by default; high only for hard debugging
Reads are most of the tool tokens	File reads were 90.6% of tool-result tokens, shell output 6.4%, in one measured workload (Marmelab)	Search first, read line ranges, navigate by symbol

Fact	Evidence	What it means for you
Fan-out multiplies cost	Agent teams use about 7x the tokens of one session (Anthropic); each fresh subagent writes about 14K tokens before its first call (dotclaude)	Few subagents, cheap models for them, no teams for routine work
Idle sessions still spend	Scheduled `/loop` tasks, goal check-ins, cross-session messages, recaps and prompt suggestions all send requests (Anthropic's cost guide)	Switch them off or stop them before a break
The cache expires	One hour on a subscription, five minutes on usage credits; the first message after that re-processes everything (Anthropic)	After a long break, start fresh from a short handoff
Most add-on savers do not save	Median third-party cost change: RTK +7%, Context Mode +33% to +72%, Headroom +32%; terse rules −8%, Ponytail −10%, LSP navigation −13%	Install few tools; prefer settings and habits

How this was researched. The sources were the official documentation of all four tools, eight public paired benchmarks, a catalogue of 45 token-saving tools, JetBrains' controlled tests, one developer's measured heavy week, Reddit, dev.to and Hacker News threads, YouTube walkthroughs, and the 20 screenshot pages and JPGs you attached. Each tool's claim is reported as its claim. A tool is recommended only when official guidance or a third-party measurement supports it, and benchmark publishers with a stake in a tool are named. Nothing here starts from AskAI or any other kit: AskAI appears in the research list like every other tool.

Day 0: Claude Code and Codex

Install the CLI, sign in with your plan, confirm you are not on per-token billing, then apply the settings below once. The installation script in this kit does all of it, adding each key only when you have not set it yourself.

Claude Code

1. Install: `curl -fsSL https://claude.ai/install.sh | bash` (macOS, Linux, WSL) or `irm https://claude.ai/install.ps1 | iex` (Windows), then `claude` and `/login` with your Pro, Max or Team account.
2. Run `/status`. It must show your subscription, not an API key. Unset `ANTHROPIC_API_KEY` and `ANTHROPIC_BASE_URL` unless you mean to pay per token: an API key overrides your plan, and `claude -p` always uses it.
3. Apply these settings in `~/.claude/settings.json`:

Setting	Value	Why
`model`	`haiku` for simple work, `sonnet` as the everyday default	Claude Code starts on Opus otherwise; escalate per task with `/model`
`env.CLAUDE_CODE_SUBAGENT_MODEL`	`haiku`	Search and log-reading subagents rarely need more
`effortLevel`	`medium`	Thinking tokens are billed as output
`autoCompactWindow`	`150000`	Turns above 150K were three quarters of one heavy week's cost
`promptSuggestionEnabled`, `awaySummaryEnabled`	`false`	Each sends a request you did not ask for
`crossSessionInbound`	`"hold"`	A message from another session does not wake this one
`env.CLAUDE_CODE_GOAL_CHECKIN_MINUTES`	`"0"`	No idle goal check-ins re-sending the full context
`bashOutputMaxChars`	`15000`	Caps shell output read back into context (default 30,000)

Setting	Value	Why
`permissions.deny`	`Read()` of `node_modules`, `dist`, `build`, lockfiles, `*.min.js`, `*.map`, `.env`	Stops the most expensive useless reads, and keeps secrets out
`includeGitInstructions`	`false`, only if you do not use Claude for commits	Removes commit and PR instructions plus the git snapshot from every turn

4. Keep `~/.claude/CLAUDE.md` under 25 lines of rules that change behaviour: read line ranges after a search, batch independent reads, make the smallest change that solves the task, answer tersely, limit subagent tool calls, and a `# Compact instructions` section saying what to keep. Project detail goes in the repo's `CLAUDE.md` (under 200 lines) or in skills that load on demand.
5. Optional, with evidence: Claude Code's official code-intelligence plugins for your languages, for example `npm i -g typescript-language-server typescript` then `/plugin install typescript-lsp@claude-plugins-official` (also `pyright-lsp`, `kotlin-lsp`, `ruby-lsp`, `swift-lsp`). One benchmark measured LSP navigation at −13% cost.
6. Measurement: `npm i -g ccusage`, then `ccusage daily` and `ccusage blocks --live`. Use `/usage` inside Claude Code for plan bars and attribution by skill, subagent and MCP server.
7. Prefer CLIs to MCP servers: install `gh`, `rg` and `jq`, and leave MCP servers disabled until a task needs one.

Codex

1. Install: `npm i -g @openai/codex`, then `codex` and sign in with ChatGPT. Check `codex login status`; unset `OPENAI_API_KEY` unless you want API billing.
2. Add to `~/.codex/config.toml` (only keys you have not set):

```
model_reasoning_effort = "medium"
model_verbosity = "low"
personality = "none"

[agents]
default_subagent_reasoning_effort = "medium"

[tui]
auto_recap = false
```

3. Use a cheaper model for routine work. On Plus, a Luna-class model allows roughly 350–3,000 messages per five-hour window against 15–160 for Sol, by published estimates. Inputs past 272K tokens bill input at twice the rate on GPT-6 models.
4. Keep `~/.codex/AGENTS.md` to the same short rules as above, and nest project `AGENTS.md` files by folder so a service's rules load only there. Codex embeds at most 32 KiB of them.
5. Leave Fast and Ultrafast modes off; they spend allowance about 2.5x and 8x faster. Run small jobs locally: OpenAI says cloud tasks can use more of the allowance.

Not worth installing for token savings

Tool	Why not
RTK	Median +7% task cost across six third-party studies; its own counter reported 96M tokens saved in runs that cost 7.6% more
Context Mode	+33% to +72% cost in the one paired study
Headroom, lean-ctx	Median +32% and +10%
Proxies and routers (ClaudeSlim, 9Router, OmniRoute, claude-code-router)	Can move you to per-token billing, break caching, and send code to third parties
Large always-on `CLAUDE.md` templates, memory databases	Their text loads into every session and subagent

Day 0: Antigravity and Cursor

Both tools meter differently from Claude Code and Codex: Antigravity by quota per model, Cursor by dollars of included usage at API rates. The cheap-model habit matters even more here.

Antigravity CLI and app

1. Install: `curl -fsSL https://antigravity.google/cli/install.sh | bash` or, on Windows, `irm https://antigravity.google/cli/install.ps1 | iex`. Run `agy` and sign in with your Google AI plan.
2. In `~/.gemini/antigravity-cli/settings.json` keep `"useG1Credits": false`, and in the app set **AI Credit Overages** to **Never**, so paid credits are not spent silently when a quota runs out.
3. Pick models per quota: quota is tracked per model, and Flash has its own pool that refreshes more often than Pro. Use Flash for routine work, Pro or Claude for hard tasks, and check `/usage` (`/quota`) before a long job.
4. Keep `~/.gemini/GEMINI.md` short. All always-on rules share a 20,000-token budget, and each rule file is cut at 24 KB. Put detailed rules in `~/.gemini/config/rules/*.md` or the repo's `.agents/rules/` with `trigger: glob` (loads only for matching files) or `trigger: model_decision` (only a one-line description loads until needed).
5. Put repeatable workflows in skills (`~/.gemini/antigravity-cli/skills/<name>/SKILL.md`). Only their name and description load until used, and each becomes a slash command.
6. MCP servers live in `~/.gemini/config/mcp_config.json`. Add only what a project needs, and use `"disabled": true` or `"disabledTools"` to trim their tool lists.

Cursor

1. Install the editor, and for the terminal agent run `curl https://cursor.com/install -fsS | bash`. Sign in.
2. Use **Auto** for routine work. Cursor's current pricing page says Auto draws on your included usage at $1.25 per million input tokens, $6 per million output and $0.25 per million cache reads. That is far cheaper than a frontier model, but not free; older community posts saying Auto does not count are out of date.
3. Leave **Max Mode** off. It raises the context window up to 1M tokens and costs more per request. Choose a frontier model only for the hard task, then switch back.
4. Add a `.cursorignore` to each repo covering `node_modules`, build output, lockfiles, `.env*` and large data files, so the agent never indexes or reads them.
5. Rules: one short always-on user rule with the same token rules as above, and project rules in `.cursor/rules/*.mdc` with `globs` and `alwaysApply: false`, instead of one large `.cursorrules` file.
6. Watch the usage dashboard (`cursor.com/dashboard`, Usage tab), which lists tokens per request, and use Tab completions for small edits: they are unlimited on paid plans.

While working: commands to keep handy

The same dozen moves exist in all four tools under slightly different names. Every command below is from each vendor's reference as of October 2026; run `/help` after an upgrade.

Need	Claude Code	Codex CLI	Antigravity CLI	Cursor
What fills my context	`/context`	`/status`	`/context`	`/context`
Usage left, and where it went	`/usage` (by skill, subagent, MCP; `d` / `w`)	`/usage daily` / `weekly`, `/status`	`/usage` (`/quota`), `/credits`	Dashboard, Usage tab
New task, fresh chat	`/clear`	`/new` (`/clear`)	`/clear` (`/new`)	`/clear` (`/new`)
Shrink a chat you must keep	`/compact keep: goal, files, failing test`	`/compact`	fresh chat + handoff	`/summarize` (`/compress`)
Summary without compacting	`/recap`	`/recap`	none	none
Stop a wrong turn	`Esc`, then `/rewind` or `Esc Esc`	`Esc`, then `Esc Esc` (edit and fork)	`Esc`, then `/rewind` (`/undo`)	`Esc`, then `/rewind`
Try an idea on a branch	`/rewind`, then retry	`/fork`	`/fork` (`/branch`)	`/fork`
Plan before editing	`Shift+Tab` or `/plan`	`/plan`	`/plan`, `/planning`	`/plan`, `/ask`
Side question, no derail	`/btw`	`/side` (`/btw`)	`/btw`	`/ask`
Change model	`/model`	`/model`	`/model`	model picker
Lower thinking	`/effort low` or `medium`	`Alt+,` (raise: `Alt+.`)	`/effort low`	pick a non-thinking model
Strong model only at decision points	`/advisor opus` with a Sonnet session	none	none	none

Need	Claude Code	Codex CLI	Antigravity CLI	Cursor
Turn off unused tools	`/mcp`	`/mcp`	`/mcp`	Settings, MCP
Point at one file	`@path`	`@path`	`@path`	`@path`
Resume later	`/rename`, then `/resume`; `claude -c`	`codex resume --last`	`/resume` (`/switch`)	`agent --continue`
Handoff before a break (kit)	`/handoff`	`$handoff`	`/handoff`	`/handoff`
Health and config check	`/doctor`	`/status`	`/help`, `agy --version`	`/about`

Outside the agents: `ccusage daily` and `ccusage blocks --live` for Claude Code usage over time, and `/insights` in Claude Code for a report on your habits. `/insights` costs tokens itself, so run it now and then, not daily.

While working: etiquette that saves the most

The habits matter more than any tool, because every turn re-reads the context and every wrong path is paid for twice. They apply to all four tools.

When	Rule	Evidence
Before you type	One task per chat; `/clear` between unrelated tasks. `/clear` is free; `/compact` is itself a large request	Anthropic cost guide; your screenshot page 19
	Send one complete prompt: goal, the files, and how to check it is done. Do not drip-feed follow-ups	Each turn re-reads everything (dotclaude); page 1; AI LABS video
	Name files and functions with `@path`: "add validation to `login()` in `auth.ts`", not "improve the codebase"	Anthropic; Antigravity best practices; agentpedia
	Plan first for anything touching more than two files, and approve before edits	Anthropic, Antigravity, Cursor docs; page 2

When	Rule	Evidence
	Give a verification target: a test, a command, an expected output	Anthropic and Antigravity best practices
	Ask for one round of clarifying questions, not open-ended "95% confidence" loops	Page 2's rule, kept to one round so it does not add turns
While it works	Stop wrong turns early with `Esc`, then rewind instead of arguing	All four vendors' docs
	If an answer is slightly wrong, edit your message and regenerate rather than adding a correction	Page 1: follow-ups stay in history
	Paste only the failing function or the last 10–40 log lines	Page 5; Anthropic's hook example filters test output to failures
	Search, then read line ranges; batch independent reads in one message	Reads are about 90% of tool tokens (Marmelab); parallel calls cost one turn (dotclaude)
	Ask for the smallest change that solves the task, with no unrequested refactors	Ponytail's idea, −10.3% cost (JetBrains); Karpathy guidelines, page 13
Context and tools	Keep `CLAUDE.md` / `AGENTS.md` / `GEMINI.md` under 200 lines, as an index that points to detail	Anthropic; page 6; Codex caps at 32 KiB; Antigravity rule budget
	Disable MCP servers you do not use today; prefer `gh`, `rg`, `jq`	Anthropic cost guide; page 20 (its 17,600-tokens figure predates tool deferral)
	Convert PDFs and HTML to Markdown before attaching (for example with `markitdown`)	The slide series' public posts; about 90% fewer tokens claimed
	Check `/context`; at a natural boundary either compact with instructions or, better, hand off and `/clear`	Page 7 says compact at ~60% and clear after 3–4 compacts
Models and agents	Cheapest model that can do the job; escalate after a failure, then go back down	The only repeated large saving in the benchmarks

When	Rule	Evidence
	Few subagents, on a cheap model; no agent teams for routine work	Teams ~7x (Anthropic); page 8; each subagent writes ~14K tokens first
	Instead of switching the whole session to Opus, try Sonnet with `/advisor opus`	Advisor is consulted only at decision points
Money traps	Leave speed and big-context modes off: Claude fast mode (credits at 2x), Codex Fast and Ultrafast, Cursor Max Mode	Each vendor's pricing docs
	Run big jobs outside Claude's peak hours: weekdays 08:00–14:00 US Eastern, which is 17:30–23:30 IST until early November and 18:30–00:30 IST after	Anthropic, March 2026; page 9
	If usage jumps while idle, compare `/stats` in the CLI with claude.ai's usage bar	Reported counter-desync bug (GitHub issues #38335, #39507)
	Judge any tool by accepted, tested tasks per subscription, not by its own savings counter	RTK reported 96M tokens saved in runs that cost 7.6% more (JetBrains)

Session breaks: before, during, after

A break costs tokens in two ways: background work keeps spending while you are away, and the first message after the cache expires re-processes the whole conversation. A 40-line handoff file and a fresh chat avoid both.

Before you step away

- [ ] Finish the current verified step; do not start a new subtask.
- [ ] Run the focused test, then commit or stash.
- [ ] Write the handoff: `/handoff` in Claude Code and Cursor, `$handoff` in Codex, `/handoff` in Antigravity (the kit installs all four). It overwrites `.ai-work/HANDOFF.md` with goal, verified state, branch and commit, decisions, next steps and the files to read first.

- [ ] Stop background work: scheduled `/loop` tasks, `/goal`, agent teams and background agents in Claude Code; `/goal pause` in Codex; background subagents under `/agents` in Antigravity; cloud agents in Cursor.
- [ ] Name the session (`/rename` in Claude Code) and note when your window resets (`/usage`, `/status` or the dashboard).

While you are away

Nothing should be running. Claude Code compacts idle conversations over 200K tokens on its own in recent versions, but a stopped session costs nothing at all.

When you come back

Situation	What to do
Back within the cache lifetime (1 hour on a Claude subscription, 5 minutes on usage credits) and the context is small	Continue in the same chat
Longer break, or the context is past ~100K tokens	Fresh chat, then paste the resume prompt below
Switching to another of the four tools	The same handoff file and prompt work in any of them
Several breaks in one long session	Hand off and clear instead of compacting a third or fourth time

Resume prompt (any tool):

```
Read AGENTS.md, then .ai-work/HANDOFF.md, then only the files it lists.
Run `git status --short` and `git log -1 --oneline` and check the handoff
against them.
Summarise the state in 6 lines, confirm the next step in one sentence, and
wait for my go-ahead.
```

When you hit a limit

Tool	Options, cheapest first
Claude Code	Wait for the reset time shown. After a model-specific limit ("You've hit your Opus limit"), `/model sonnet` keeps you working. A saved limit reset is offered in claude.ai Settings, Usage (web and Desktop). `/usage-credits` buys more
Codex	`/usage` shows resets and, where offered, a banked reset. Switch to a Luna-class model, whose allowance is far larger
Antigravity	Quota is per model: switch to Flash, which has its own pool. Keep overages on Never unless you mean to buy credits
Cursor	Switch to Auto, check the dashboard for the expensive requests, and set a spend limit

Model and effort routing

Start each task on the cheapest model that can do it, escalate after one failed attempt, and come back down when the hard step is done. Switch at task boundaries: the prompt cache belongs to one model, so a mid-task switch re-reads the whole conversation at full price.

Task	Claude Code	Codex	Antigravity	Cursor
Search, explain, rename, small edits, tests, docs	Haiku, effort low	Luna-class model, effort low	Flash, effort low	Auto
Everyday features and bug fixes	Sonnet (Haiku when it copes), effort medium	Sol, effort medium	Flash, or Pro if Flash struggles, medium	Auto, or a mid-tier model
Hard debugging, architecture, security	Sonnet with `/advisor opus`, or Opus, effort high	Sol, effort high	Pro or Claude, effort high	A frontier model, still without Max Mode
Subagents and parallel workers	Haiku (`CLAUDE_CODE_SUBAGENT_MODEL=haiku`)	`default_subagent_reasoning_effort = "medium"`	Flash	Keep them few

One of your screenshots (page 13) shows a routing table that uses only Sonnet and Opus at high or max effort. That table maximises quality, not usage; keep the idea of writing your routing rules into `CLAUDE.md`, but use the cheaper rows above.

The installation script

`token-setup` applies everything in the Day 0 sections in one run, for whichever of the four tools you use. It previews by default and changes nothing until you add `--apply`.

```
# macOS, Linux, WSL
bash setup/setup.sh                                  # preview every change
bash setup/setup.sh --apply                          # apply settings, rules
and handoff commands
bash setup/setup.sh --apply --install-clis           # also install missing
CLIs and ccusage
bash setup/setup.sh --apply --project ~/code/myapp   # also add the repo
files
bash setup/setup.sh --uninstall                      # remove only what it
added
```

```
# Windows
powershell -ExecutionPolicy Bypass -File .\setup\setup.ps1            #
preview
powershell -ExecutionPolicy Bypass -File .\setup\setup.ps1 --apply
```

Tool	What it sets
Claude Code	`settings.json`: cheap default model, Haiku subagents, medium effort, auto-compact at 150K, prompt suggestions and recaps off, cross-session messages held, no idle goal check-ins, Bash output capped at 15K, 15 deny-read rules; a ccusage status line when none is set. A 12-line rules block in `CLAUDE.md` and a `/handoff` command
Codex	`config.toml`: medium effort, low verbosity, no personality section, medium subagent effort, no automatic recaps. The rules block in `AGENTS.md` and a `handoff` skill
Antigravity	`useG1Credits: false`, the rules block in `GEMINI.md`, a glob-triggered code rule, and a `handoff` skill for the CLI and the app

Tool	What it sets
Cursor	An always-on user rule with the same short rules, and a `/handoff` command
Each repo (`--project`)	`AGENTS.md` (under 60 lines), `CLAUDE.md` importing it, `.cursorignore` , `.ai-work/HANDOFF.md` , and `.ai-work/` in `.gitignore`

Option	Effect
`--tools claude,codex,agy,cursor`	Choose hosts; the default is Claude Code plus every other tool found on the machine
`--claude-model haiku|sonnet|opus|keep`	Default model when you have none set (default `haiku` ; `sonnet` for heavier everyday work)
`--lsp ts,py,kotlin,ruby,swift`	Claude Code's official code-intelligence plugins, with npm language servers for ts and py
`--lean-git`	Also drop the git instructions from every turn
`--install-clis`	Install missing CLIs: vendor installer scripts are downloaded, shown, and run only after you type `YES`

Safety. It warns when `ANTHROPIC_API_KEY` , `ANTHROPIC_BASE_URL` or `OPENAI_API_KEY` is set. It adds a key only when you have not set it, so your own `model` or effort stays as it is. It backs up each file before the first change to `~/.token-kit/backups/<time>/` and writes rule text only inside marked blocks. It never overwrites a file it did not write, and `--uninstall` removes only unchanged keys, its blocks and its own files. A re-run changes nothing. The Linux run was tested end to end (preview, apply, re-apply, uninstall); the Windows launcher runs the same Node script but was not executed on Windows.

Research: every token-saving tool, claimed vs measured

Of the 50-odd tools found, only a handful have an independent measurement that shows a saving, and none comes close to its headline. "Measured" means a third-party paired benchmark: JetBrains, THOL, Dasein Labs, Stet.sh, Tura, Marmelab or Codepointer, as collected in the cc.bruniaux.com catalogue. Negative is cheaper. Read single-study numbers as direction, not precision.

Category	Tool	What it claims	Measured by third parties	Verdict
Built in	`/usage`, `/context`, `/doctor`, plan mode, `/clear`, effort, skills, MCP tool deferral	—	Official guidance in all four vendors' docs	Use daily
Built in	Claude Code advisor (`/advisor opus`)	Opus judgment at Sonnet cost	Independent analyses: about a third cheaper than Opus throughout	Use instead of switching to Opus
Measurement	ccusage	Usage and cost from local logs	Reports only	Install
Measurement	Claude-Code-Usage-Monitor, Antigravity Cockpit (VS Code), Cursor dashboard	Live usage views	Reports only	Optional
Behaviour rules	Ponytail	−54% code, −20% cost	**−10.3% cost** (JetBrains, p=0.004); −3.5% (THOL); −8.9% (Tura); +20% then −2% (Stet.sh)	Use, as a skill or its rule text
Behaviour rules	Caveman skill	65% fewer tokens	**Median −8% cost** over six studies; quality unchanged	Use the skill or a terse rule; skip its proxy
Behaviour rules	claude-token-efficient	63% fewer words	−11.6% (THOL); its own benchmark 4–12% output	Fine as a small file

Category	Tool	What it claims	Measured by third parties	Verdict
Behaviour rules	Karpathy guidelines	Fewer rewrites	No measurement	Use the four rules; they overlap Ponytail
Behaviour rules	Honey, Governor	−24% cost; 45.5% tokens	None independent	Watch
Code navigation	Claude Code LSP plugins	Symbol navigation instead of reads	**−13% cost** (Marmelab)	Use for typed languages
Code navigation	Serena	Faster, more reliable navigation	None	Optional; works in all four tools
Code navigation	CodeGraph, code-review-graph	44% cheaper; —	−7.6%; −8.5% (THOL, agent rarely used the tool)	Optional
Code navigation	Graphify, Claude Context, Semble, Code Context Engine, codesight, Stacklit, tilth, grepai	40–99% of retrieval tokens	None, or ~0% when the agent ignored the tool	Skip
Code navigation	Token Savior	−80% tokens	Claim withdrawn by the project	Skip
Shell output filters	RTK	60–90%	**Median +7% cost** over six studies (JetBrains +7.6%, p=0.004)	Not recommended

Category	Tool	What it claims	Measured by third parties	Verdict
Shell output filters	snip, tokf, PandaFilter, Tokenade, squeez	60–90% of command output	None, except squeez −6.9% (THOL) and Tokenade on its author's own leaderboard	Skip
Context sandboxes	Context Mode	98% of output bytes	**+33% to +72% cost** (Stet.sh)	Not recommended
Proxies and compression	Headroom	20% fewer tokens	**Median +32%** over four studies; can switch Claude Code to API billing	Avoid
Proxies and compression	lean-ctx, pxpipe, llmtrim, ClaudeSlim	59–99%	lean-ctx +10.3% (THOL); others none; pxpipe is lossy	Avoid
Hosted or paid	Edgee, Fermat, Parsec, WOZCODE	25–70%	−14.7%; −22%; none; −13% (single studies)	Only after a security review
Routers	9Router, OmniRoute, claude-code-router, routatic-proxy, Weave Router	40–70% via cheaper models	None comparable	Avoid on a subscription: billing, terms and data risk
Memory	claude-mem, ICM, Greplica, Magic Compact, Dynamic Context Pruning	10x retrieval; 40–50%	None independent	Skip; use a handoff file
MCP trimming	mcp2cli, Token Optimizer MCP	96–99% of schema tokens; 0.93x cost	None independent	Only for MCP-heavy setups

Category	Tool	What it claims	Measured by third parties	Verdict
Rules on demand	CARL / BASE	Keyword-loaded rule domains	None	Use skills and glob rules instead
Documents	markitdown	PDF and Office to Markdown	None, but the mechanism is direct	Use for document work
Delegation	Spotify shunt	82–94% on large reads	Needs Spotify Portal	Not applicable
Bundles	tokless, dotclaude	Wire several of the above	Inherit their tools' results; dotclaude publishes its own measured usage	Read dotclaude's evidence
Bundles	AskAI (your project)	Installs tools by kind of work	Its own 7-day ledger found its earlier self-built hooks net negative (−2.6%); 3.1 turns RTK and Context Mode off	Optional; the script in this kit stands alone
Alternative agents	maki, Tura	Fewer tokens per turn	Vendor-run only	Not applicable

Your screenshots and popular tips, checked

Your PDF has 20 pages: 11 slides from the AI Automation Society series, plus posts from @vaibhavsisinty, @charlieautomates, @techfren, @wdf_ai and @agilesingh. Four of the five JPGs are copies of pages 8, 9, 10 and 12 (the fifth duplicates one of those). Every page was read again for this guide.

Page	What it says	Verdict
1	Batch prompts into one message; edit and regenerate instead of follow-ups	True: each turn re-reads the context

Page	What it says	Verdict
2	Plan mode first; "95% confidence, ask follow-up questions" in CLAUDE.md	True for planning; keep the questions to one round
3	`/context` and `/cost`	True; `/cost` now lives inside `/usage`
4	Set up a status line	True; ccusage or Claude Code's own fields show context and cache
5	Paste only the function or the last 10 log lines	True
6	Lean CLAUDE.md under 200 lines, used as an index	True, and Anthropic's own advice
7	Compact at about 60%; after 3–4 compacts, summarise and clear	Partly: compacting is itself a big request, so hand off and clear at a task boundary instead
8	Agent workflows use 7–10x tokens; Haiku subagents; teams very expensive	True: Anthropic cites about 7x for teams
9	Peak hours 8 AM–2 PM ET on weekdays	True for Claude (17:30–23:30 IST in US summer, 18:30–00:30 IST in winter); not documented for the other three
10–11	Headroom install (`pip install headroom-ai` , v0.24.0)	Works, but four studies put its median cost at +32%
12	`/doctor` audit; rule layers loaded by keyword; a cache-aware status line	`/doctor` is built in and worth a weekly run
13	Karpathy rules; a routing table using only Sonnet and Opus at high or max effort	Keep the rules; the routing table maximises quality, not usage
14	"3 hacks to save up to 50%": `/doctor` , an effort table in CLAUDE.md, manual effort	The habits are sound; the 50% figure is unverified
15	`/effort` and `/code-review` in Claude Code's picker	True; both are built in
16	A "Token Saver" panel toggling RTK, Headroom, Caveman, Ponytail	A router dashboard; of the four, only Caveman-style and Ponytail-style rules show savings
17	Caveman: 69 vs 19 tokens; terse commits; input compression	About 8% fewer tokens in agent work, not 65%

Page	What it says	Verdict
18	`headroom wrap claude` with Claude Code showing **API Usage Billing**	Real risk: a proxy can move you from your plan to per-token billing
19	`/clear` between unrelated tasks, the #1 habit	True, and free
20	Disconnect MCP servers (about 17,600 tokens each) and use CLIs	Direction right; the figure predates Claude Code deferring MCP tool definitions

Popular tip	Verdict
"RTK saves 60–90%"	False at task level: +7% median cost; the 60–90% is shell output only
"Caveman saves 65%"	About 8%, measured on agent tasks
"Cursor Auto is free and unlimited"	Outdated: Auto counts against your included usage, at low rates
"Delete old transcripts to save tokens"	False: they never enter the context; it only frees disk
"Idle Claude Code burns tokens"	Mostly false for a stopped session (background jobs cost under $0.04 per session); true for scheduled loops, goals and teams left running
"Use the 1M window as insurance, reset around 120K"	True: calls above 150K dominate cost
"Thinking is free"	False: thinking is billed as output
"Spend leftover quota before the 5-hour reset" (Antigravity community)	Only if your weekly cap is not the limit you hit

Sources

Read between 19 and 20 October 2026. Vendor commands and settings change often; check `/help` and each vendor's docs after an upgrade.

Official documentation

- Claude Code: Manage costs effectively, environment variables, code intelligence plugins

- Codex: sample configuration, pricing and usage FAQ, slash commands, skills
- Antigravity: CLI commands and settings, CLI best practices, rules, skills, MCP, plans and quotas, AI credits
- Cursor: pricing and Auto rates, models and Max Mode

Independent measurements

- JetBrains: RTK tested, Ponytail tested
- Token-saving tools: claims and third-party studies (45 tools, 8 benchmarks; the author contributes to RTK and says so)
- dotclaude: measured usage of a heavy week
- Marmelab: which agent plugin should you use in 2026

Community and limits

- Codex usage limits 2026, Codex vs Claude Code, October 2026
- Claude Code advisor explained, advisor vs a bigger model
- Antigravity quotas explained, Cursor usage limits explained
- Claude Code max quota consumption and the counter-desync bug

YouTube

- Stop Wasting Claude Tokens (Amine AI, 10 Oct 2026)
- How To Never Run Out Of Codex and Claude Usage Limits (AI LABS, 28 Aug 2026)
- I Tested Usage Limits for Claude, Codex, and Cursor (AI Coding Daily, 2 Sep 2026)

Your attachments: the 20-page screenshot PDF and four JPGs, checked page by page in the section above.